



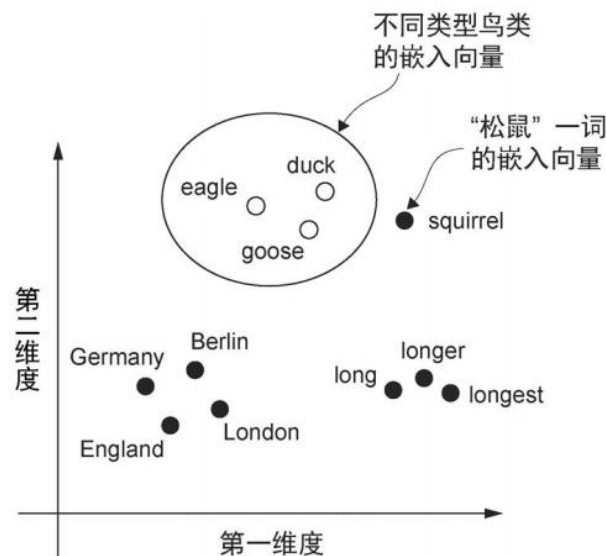
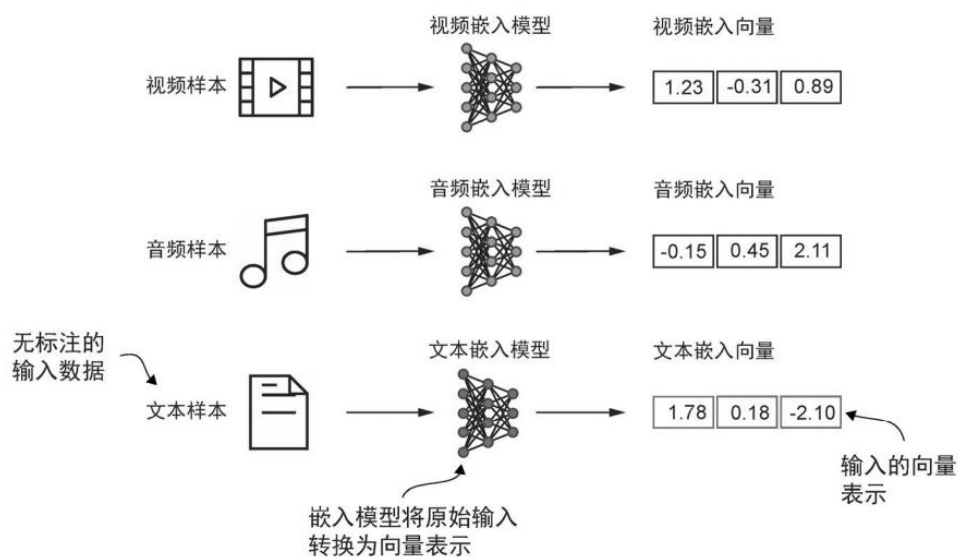
第2章： 文本数据处理

理解词嵌入 Understanding Word Embeddings

深度神经网络无法处理离散的文本数据，必须将其转换为连续值的向量。**嵌入 Embedding** 是将离散对象(单词、图像)映射到连续向量空间的过程。

高维特性： GPT-2 (small) 使用768维， GPT-3 使用12,288维。高维度有助于捕捉复杂的语义关系。

关键点：早期方法如 **Word2Vec** 是静态的，而 LLM 的嵌入层是模型的一部分，在训练过程中与模型权重一起优化。



基础文本分词 Basic Tokenization

01

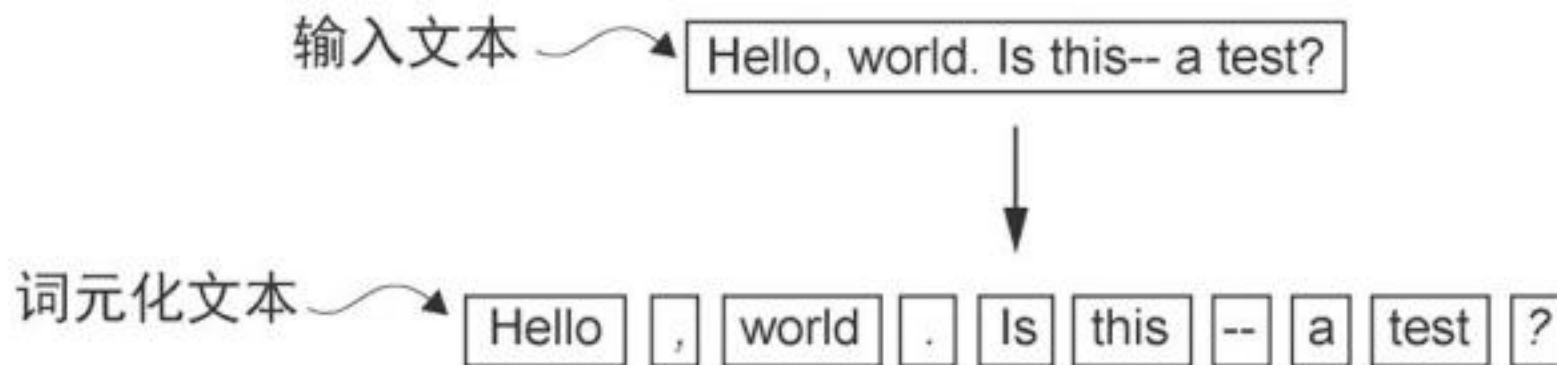
离散文本 vs 连续计算

- **文本 (Text):** 人类可读的符号 (如 "Life is short")
- **词元 ID (Token IDs):** 将符号映射为整数 (如 [45, 12, 99])
- **嵌入向量 (Embeddings):** 将整数映射为连续的浮点数向量 (如 [0.12, -0.45, ...])

02

基础分词实现

- **标准化:** 读取文本, 处理标点符号
- **拆分 (Split):** 使用 正则表达式将文本切分为列表
- **去重与排序:** 提取所有唯一的单词
- **构建映射表:** 词 —— ID



构建词汇表与 ID 映射 Building the Vocabulary

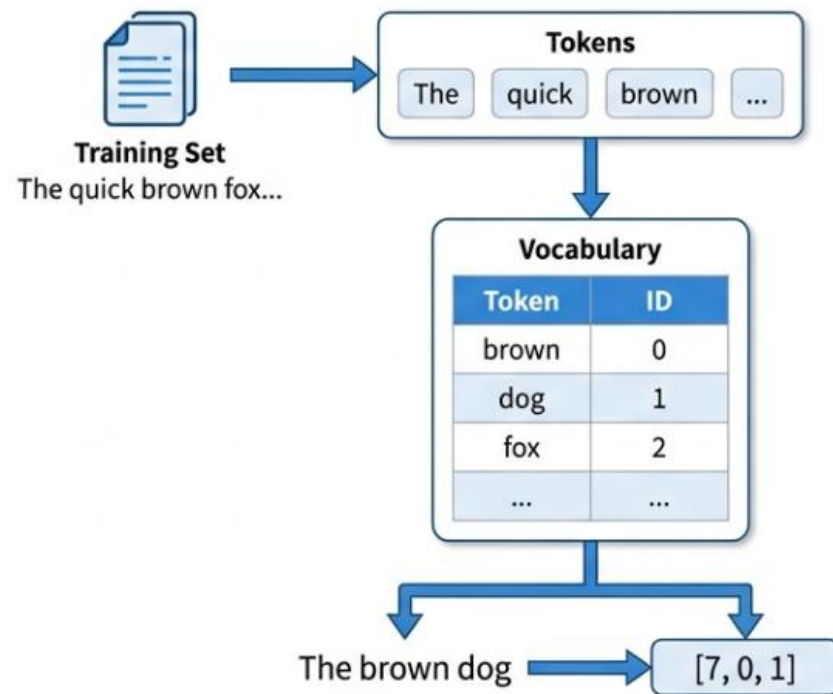
Process

为了让模型理解文本，我们需要将文本词元 (String) 映射为唯一的整数 ID (Integer)。

1. **Extract:** 获取训练集中所有唯一的词元 `set(preprocessed)`
2. **Sort:** 按字母顺序排列 `sorted( )`
3. **Map:** 创建 token 到 integer 的映射字典

```
all_words = sorted(set(preprocessed))  
vocab = {token:integer for integer, token in enumerate(all_words)}
```

对于示例文本“The Verdict”，词汇表大小为1130个唯一词元。

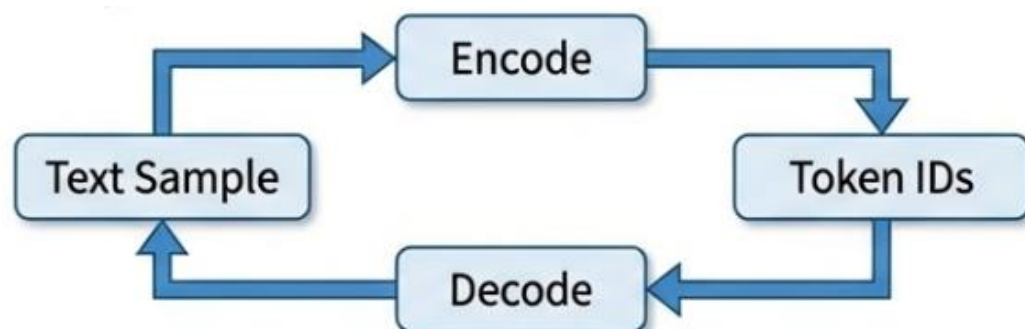


实现简易分词器 Implementing SimpleTokenizerV1

类逻辑 Class Logic

我们将封装一个Python 类来处理编码 Encode 和解码 Decode。

- **__init__** : 存储正向映射 (str_to_int) 和反向映射 (int_to_str)
- **encode(text)**: 文本 -> 分词 -> ID 列表
- **decode(ids)**: ID列表->文本词元->拼接字符串



实现 Implementation

```
class SimpleTokenizerV1:
    def __init__(self, vocab):
        self.str_to_int = vocab
        self.int_to_str = {i:s for s,i in vocab.items()}

    def encode(self, text):
        preprocessed = re.split(r'([,.;?!"()\'|]|--|\s)', text)

        preprocessed = [
            item.strip() for item in preprocessed if item.strip()
        ]
        ids = [self.str_to_int[s] for s in preprocessed]
        return ids

    def decode(self, ids):
        text = " ".join([self.int_to_str[i] for i in ids])
        text = re.sub(r'\s+([,.;?!"()\'|])', r'\1', text)
        return text
```

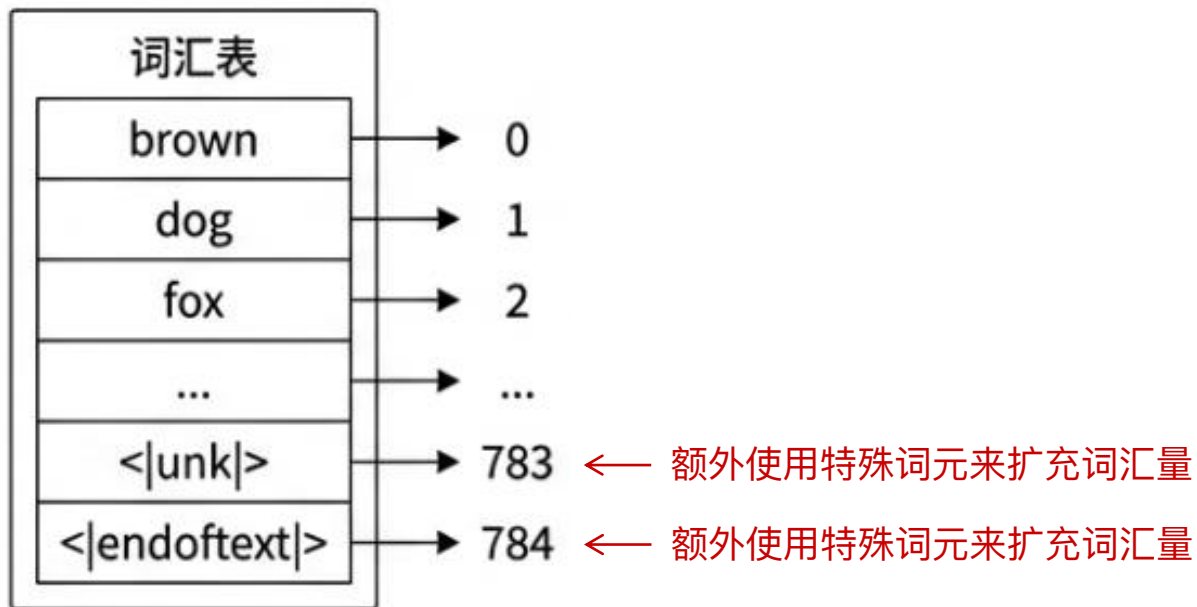

引入特殊上下文词元 Special Context Tokens

基础分词器无法处理未知单词，且无法区分不同的文档来源。我们需要引入特殊标记：

<|unk|>: 代表未知单词 (Unknown)，解决 KeyError 问题。

<|endoftext|>: 文档分隔符。告诉模型两个文本片段是不相关的(例如两篇不同的文章)。

词汇表示例



SimpleTokenizerV2 更新

```
# 添加特殊词元到词汇表
all_tokens = sorted(list(set(preprocessed)))
all_tokens.extend(["<|endoftext|>", "<|unk|>"])

# Encode逻辑修改, 如果词不在词汇表中, 替换为<|unk|>
preprocessed = [
    item if item in self.str_to_int
    else "<|unk|>" for item in preprocessed
]
```

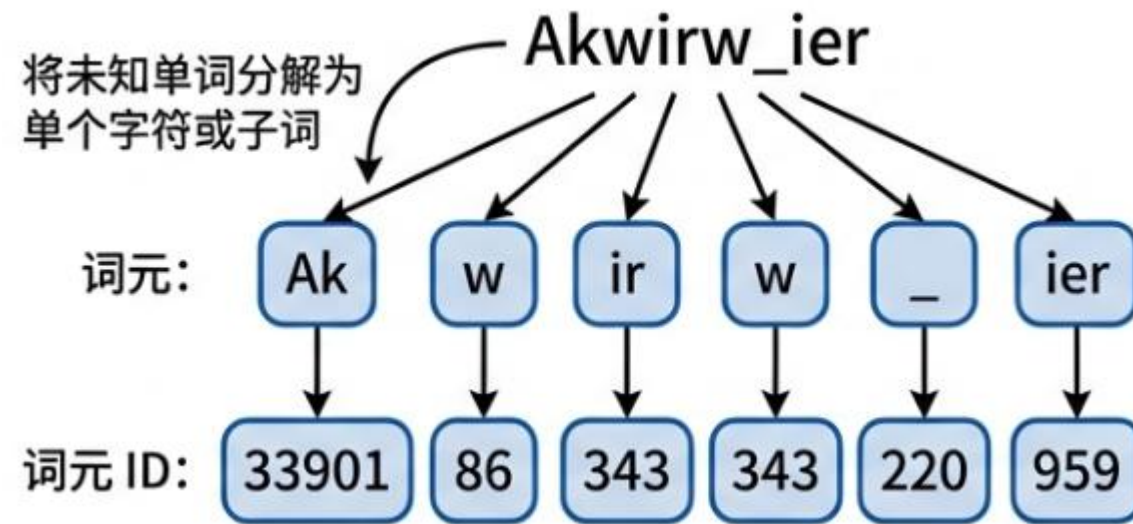
字节对编码 Byte Pair Encoding – BPE

现代大模型(如GPT-2,GPT-3,GPT-4) 使用BPE算法, 而不是基于单词的分词。

BPE 的优势在于遇到未见过的新词时, 可以将其拆解为已知的**子词 (subwords)** 甚至单个字符来表示, 因此通常不需要 `<|unk|>` 词元, 也能稳定处理任意输入文本。

OpenAI 的 `tiktoken` 库提供了高效的分词实现 (底层为 Rust), 便于在训练与推理阶段快速完成文本到 token 的转换。

以 GPT-2 为例, 其词汇表大小为 50,257 个词元。



```
import tiktoken
tokenizer = tiktoken.get_encoding("gpt2")

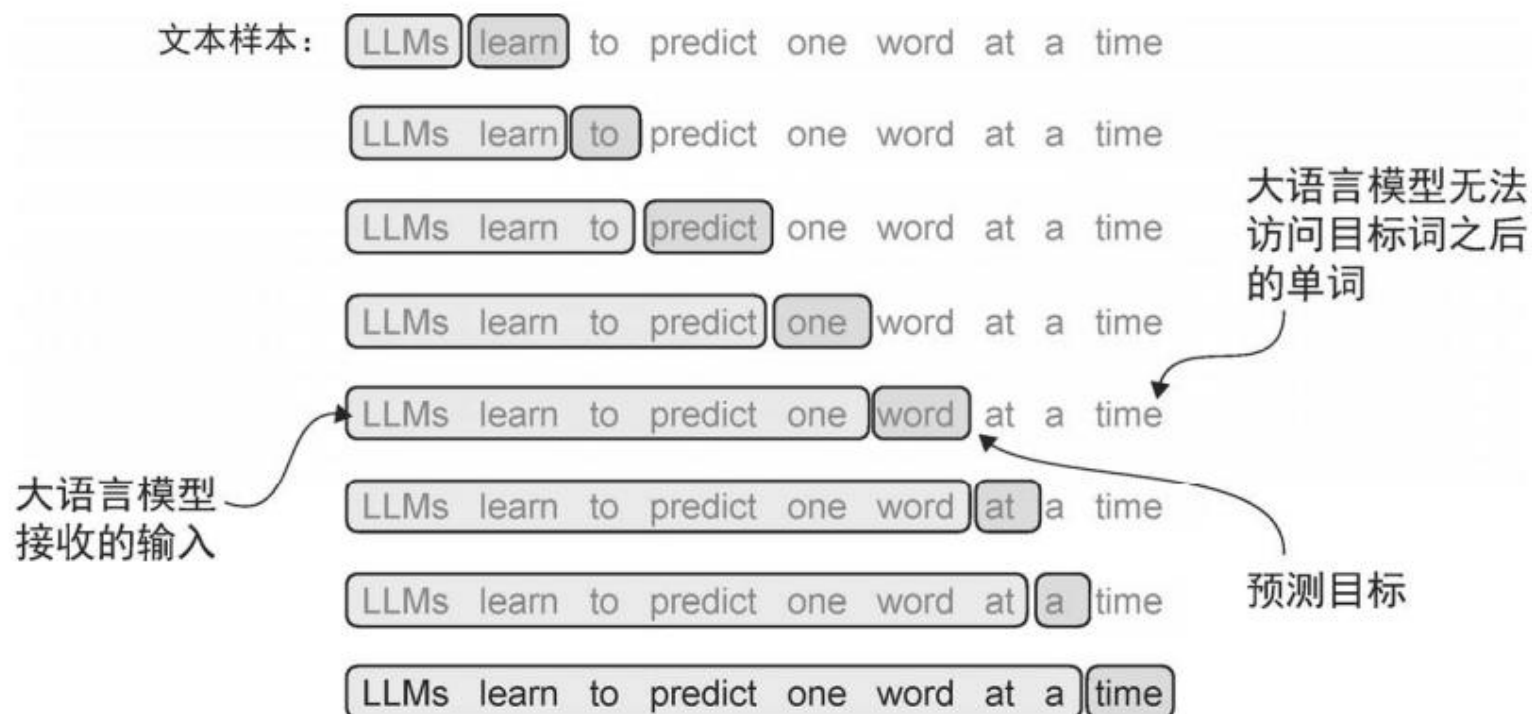
text = (
    "Hello, do you like tea? <|endoftext|> In the sunlit terraces"
    "of someunknownPlace."
)

# 允许特殊词元存在
integers = tokenizer.encode(text, allowed_special={"<|endoftext|>"})
```

滑动窗口采样 Sliding Window Data Sampling

生成 **输入 - 目标** 对 Creating Input - Target Pairs

我们要训练模型进行“下一词预测” (Next-token prediction)。给定一个输入窗口 (x)，目标 (y) 是向右移动一个位置的序列。



Example

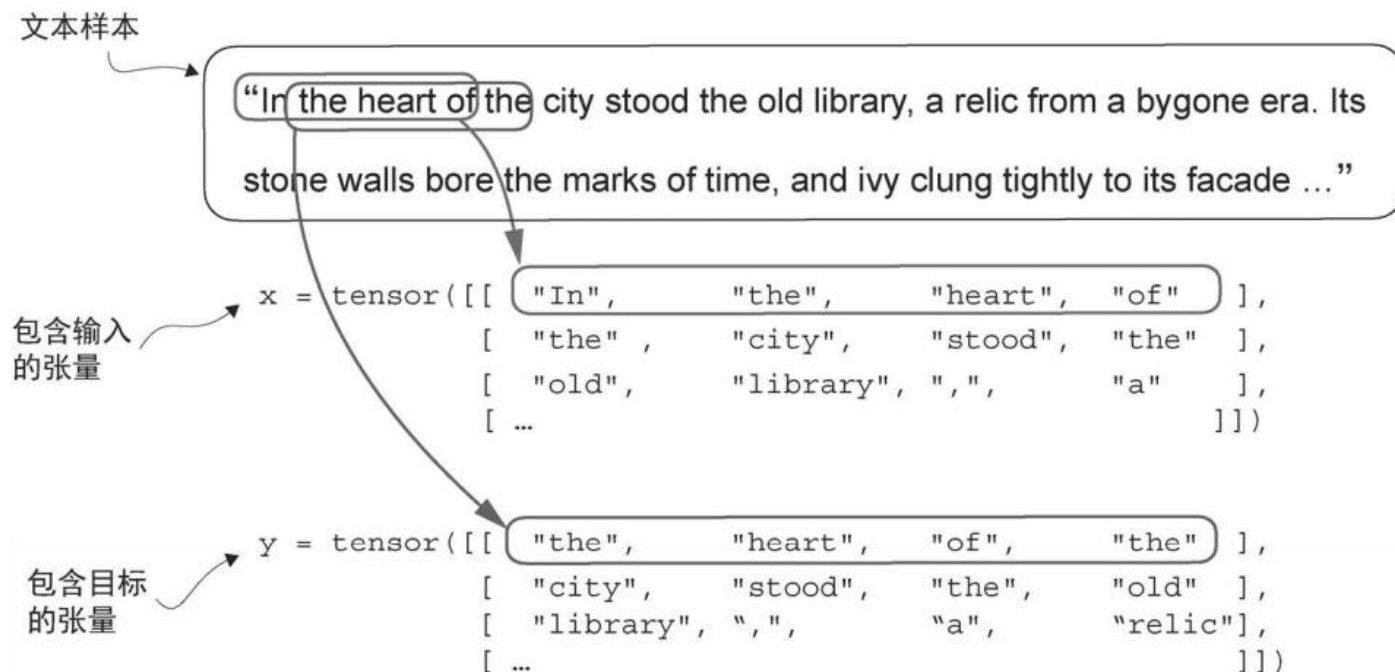
Context : [290, 4920, 2241, 287] ("and established himself in")

Target: [4920, 2241, 287, 257] ("established himself in a")

数据加载与批处理 DataLoader & Batching

核心逻辑

使用 `DataLoader` 将多个样本堆叠为批次 (Batch)



窗口大小 (Context Length / Block Size):

模型一次能“看见”的最大长度（例如 GPT-2 是 1024，本例假设为 4）。

步长 (Stride): 窗口移动的距离。

- **Stride = 1:** 最大化数据利用率，但计算量大，数据高度重叠。
- **Stride = Context Length:** 数据无重叠，训练速度快，但可能丢失跨边界的语义。
- **最佳实践:** 通常 Stride 设置这就等于 Context Length 以避免过拟合，但在数据稀缺时可减小 Stride。

创建词元嵌入层 Creating Token Embeddings

嵌入层本质上是一个可训练的查找表

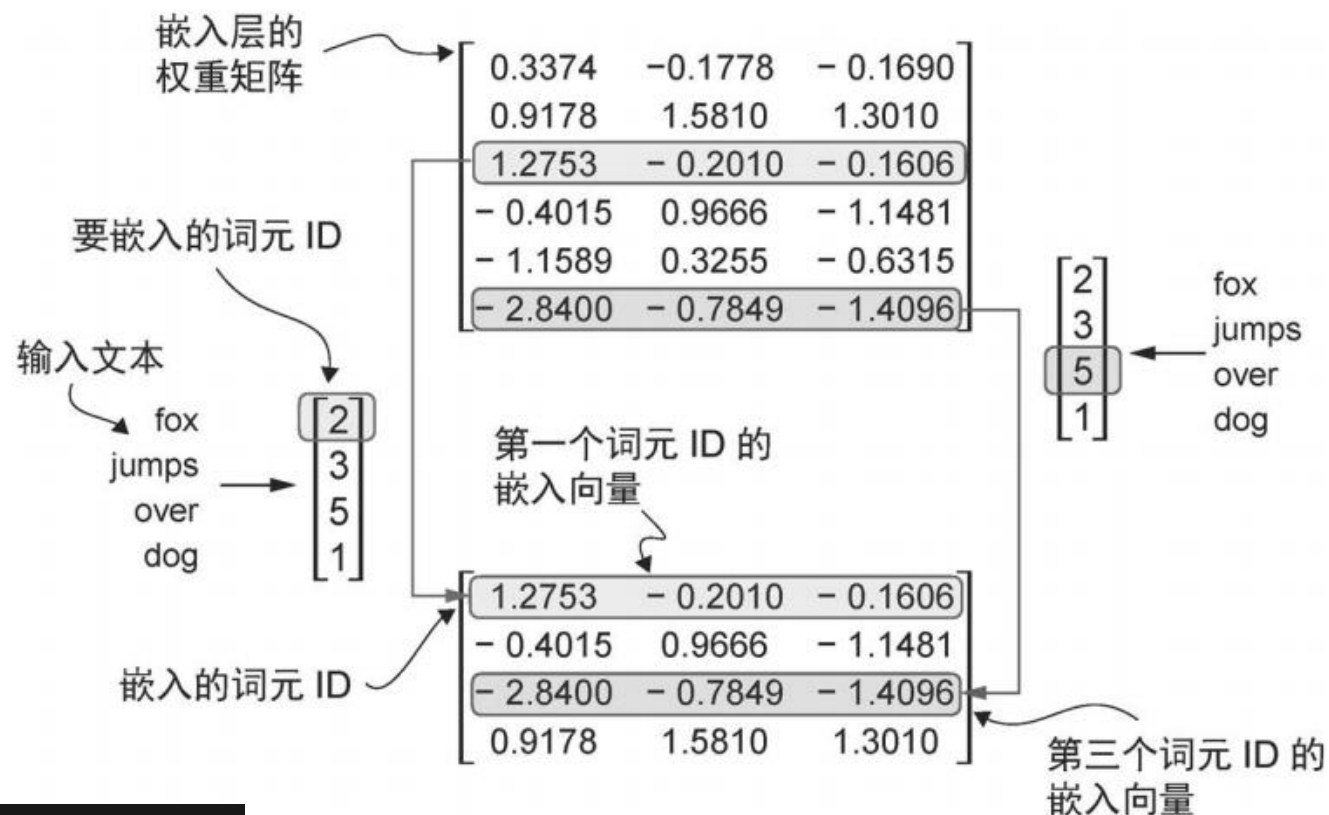
Details:

Rows = Vocabulary Size (e.g., 50,257)

Columns = Embedding Dimension
(e.g., 256 or 768)

给定一个Token ID, 层直接返回对应的行向量

这比One-Hot 编码加矩阵乘法更高效



```
vocab_size = 50257
output_dim = 256

# 初始化嵌入层
token_embedding_layer = torch.nn.Embedding(num_embeddings=vocab_size, embedding_dim=output_dim)

# 输入ID [4,256] --> 嵌入向量 [4,256,256]
```

实现绝对位置嵌入 Absolute Positional Embeddings



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

我们创建第二个Embedding 层，专门用于编码位置索引。

```
context_length = max_length

# 创建位置嵌入层：把位置索引(0~context_length-1)映射成 output_dim 维向量
pos_embedding_layer = torch.nn.Embedding(context_length, output_dim)

# 生成当前序列每个位置的嵌入向量，形状为 [max_length, output_dim]
pos_embeddings = pos_embedding_layer(torch.arange(max_length))

# 将词元嵌入与位置嵌入相加，得到最终输入嵌入（包含词义与位置信息）
input_embeddings = token_embeddings + pos_embeddings
```

词元嵌入提供了“**是什么词**”的信息。

位置嵌入提供了“**在哪个位置**”的信息。

位置嵌入被添加到词元嵌入向量中，从而生成大语言模型的输入嵌入。

位置向量与原始词元嵌入的维度相同。

